

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Суховило В. С.

Проверила:

Андренко К. В.

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
18	CIFAR-10	32X32	RMSprop

Код программы:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
import random

from PIL import Image
from collections import defaultdict

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)
```

```
if torch.cuda.is_available():  
    torch.cuda.manual_seed_all(SEED)
```

```
device = torch.device(  
    "cuda" if torch.cuda.is_available() else "cpu"  
)
```

```
CLASSES = (  
    "airplane",  
    "automobile",  
    "bird",  
    "cat",  
    "deer",  
    "dog",  
    "frog",  
    "horse",  
    "ship",  
    "truck"  
)
```

```
MEAN = (  
    0.4914,  
    0.4822,  
    0.4465  
)
```

```
STD = (  
    0.2470,  
    0.2435,  
    0.2616  
)
```

```
train_transform = transforms.Compose([  
    transforms.RandomHorizontalFlip(),  
    transforms.ToTensor(),  
    transforms.Normalize(MEAN, STD)  
)
```

```
test_transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize(MEAN, STD)  
)
```

```
train_set = torchvision.datasets.CIFAR10(  
    root="./data",  
    train=True,
```

```
    download=True,  
    transform=train_transform  
)
```

```
test_set = torchvision.datasets.CIFAR10(  
    root="./data",  
    train=False,  
    download=True,  
    transform=test_transform  
)
```

```
BATCH_SIZE = 128
```

```
train_loader = DataLoader(  
    train_set,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
    num_workers=0  
)
```

```
test_loader = DataLoader(  
    test_set,  
    batch_size=256,  
    shuffle=False,  
    num_workers=0  
)
```

```
image_shape = train_set[0][0].shape
```

```
print("Параметры программы")  
print(f"Устройство: {str(device).upper()}")  
print(  
    f"Обучающая выборка: "  
    f"{len(train_set)} изображений"  
)  
print(  
    f"Тестовая выборка: "  
    f"{len(test_set)} изображений"  
)  
print(  
    f"Размер изображения: "  
    f"{image_shape[1]} x {image_shape[2]}"  
)  
print(  
    f"Количество каналов: "  
    f"{image_shape[0]}"  
)  
print()
```

```

class CIFAR10CNN(nn.Module):

    def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(
            3,
            32,
            kernel_size=3,
            padding=1
        )

        self.conv2 = nn.Conv2d(
            32,
            64,
            kernel_size=3,
            padding=1
        )

        self.conv3 = nn.Conv2d(
            64,
            128,
            kernel_size=3,
            padding=1
        )

        self.pool = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        self.fc1 = nn.Linear(
            128 * 4 * 4,
            256
        )

        self.dropout = nn.Dropout(0.4)

        self.fc2 = nn.Linear(
            256,
            num_classes
        )

    def forward(self, x):

        x = self.pool(
            F.relu(self.conv1(x))
        )

        x = self.pool(
            F.relu(self.conv2(x))
        )

```

```

        x = self.pool(
            F.relu(self.conv3(x))
        )

        x = torch.flatten(x, 1)

        x = F.relu(
            self.fc1(x)
        )

        x = self.dropout(x)

        x = self.fc2(x)

        return x

model = CIFAR10CNN().to(device)

parameters_count = sum(
    parameter.numel()
    for parameter in model.parameters()
    if parameter.requires_grad
)

print("Архитектура нейронной сети")
print("Вход 3x32x32")
print("Conv2d 32")
print("Conv2d 64")
print("Conv2d 128")
print("Полносвязный слой 256")
print("Выход 10 классов")
print(
    f"Обучаемых параметров: "
    f"{parameters_count}"
)
print()

criterion = nn.CrossEntropyLoss()

optimizer = optim.RMSprop(
    model.parameters(),
    lr=0.001
)

N_EPOCHS = 8

```

```
history = {  
    "train_loss": [],  
    "train_acc": [],  
    "test_loss": [],  
    "test_acc": []  
}
```

```
def process_epoch(loader, training):
```

```
    if training:  
        model.train()  
    else:  
        model.eval()
```

```
    loss_sum = 0.0  
    correct_predictions = 0  
    total_images = 0
```

```
    if training:  
        context = torch.enable_grad()  
    else:  
        context = torch.no_grad()
```

```
    with context:
```

```
        for images, labels in loader:
```

```
            images = images.to(device)  
            labels = labels.to(device)
```

```
            if training:  
                optimizer.zero_grad()
```

```
            outputs = model(images)
```

```
            loss = criterion(  
                outputs,  
                labels  
            )
```

```
            if training:  
                loss.backward()  
                optimizer.step()
```

```
            loss_sum += (  
                loss.item()  
                * images.size(0)  
            )
```

```
            predictions = outputs.argmax(  
                dim=1  
            )
```

```

        correct_predictions += (
            predictions == labels
        ).sum().item()

        total_images += labels.size(0)

    average_loss = (
        loss_sum / total_images
    )

    accuracy = (
        correct_predictions
        / total_images
    )

    return average_loss, accuracy

print("Обучение нейронной сети")

for epoch in range(
    1,
    N_EPOCHS + 1
):

    train_loss, train_acc = process_epoch(
        train_loader,
        True
    )

    test_loss, test_acc = process_epoch(
        test_loader,
        False
    )

    history["train_loss"].append(
        train_loss
    )

    history["train_acc"].append(
        train_acc
    )

    history["test_loss"].append(
        test_loss
    )

    history["test_acc"].append(
        test_acc
    )

    print(
        f"Эпоха {epoch} из {N_EPOCHS} "
```

```

        f"Ошибка обучения {train_loss:.4f} "
        f"Точность обучения "
        f"{train_acc * 100:.2f}% "
        f"Ошибка теста {test_loss:.4f} "
        f"Точность теста "
        f"{test_acc * 100:.2f}%"
    )

print()

print("Результат обучения")

print(
    f"Итоговая точность "
    f"на тестовой выборке: "
    f"{history['test_acc'][-1] * 100:.2f}%"
)

epochs = range(
    1,
    N_EPOCHS + 1
)

fig, axes = plt.subplots(
    1,
    2,
    figsize=(13, 5)
)

axes[0].plot(
    epochs,
    history["train_loss"],
    marker="o",
    markersize=3,
    label="Обучение"
)

axes[0].plot(
    epochs,
    history["test_loss"],
    marker="o",
    markersize=3,
    label="Тест"
)

axes[0].set_title(
    "Ошибка модели"
)

axes[0].set_xlabel(

```

```

    "Эпоха"
)

axes[0].set_ylabel(
    "Ошибка"
)

axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(
    epochs,
    [
        value * 100
        for value
        in history["train_acc"]
    ],
    marker="o",
    markersize=3,
    label="Обучение"
)

axes[1].plot(
    epochs,
    [
        value * 100
        for value
        in history["test_acc"]
    ],
    marker="o",
    markersize=3,
    label="Тест"
)

axes[1].set_title(
    "Точность модели"
)

axes[1].set_xlabel(
    "Эпоха"
)

axes[1].set_ylabel(
    "Точность, %"
)

axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

```

```
model.eval()
```

```
class_correct = defaultdict(int)
```

```
class_total = defaultdict(int)
```

```
with torch.no_grad():
```

```
    for images, labels in test_loader:
```

```
        images = images.to(device)
```

```
        labels = labels.to(device)
```

```
        outputs = model(images)
```

```
        predictions = outputs.argmax(
            dim=1
        )
```

```
        for label, prediction in zip(
            labels,
            predictions
        ):
```

```
            label_index = label.item()
```

```
            prediction_index = (
                prediction.item()
            )
```

```
            class_total[
                label_index
            ] += 1
```

```
            if (
                label_index
                == prediction_index
            ):
```

```
                class_correct[
                    label_index
                ] += 1
```

```
print()
```

```
print("Точность распознавания классов")
```

```
for index, class_name in enumerate(
    CLASSES
):
```

```
    accuracy = (
        class_correct[index]
        / class_total[index]
        * 100
```

```
)
```

```
print(  
    f"{class_name}: "  
    f"{accuracy:.2f}%"  
)
```

```
def show_test_example(index=None):
```

```
    model.eval()
```

```
    if index is None:  
        index = random.randrange(  
            len(test_set)  
        )
```

```
    image, true_label = (  
        test_set[index]  
    )
```

```
    input_tensor = (  
        image  
        .unsqueeze(0)  
        .to(device)  
    )
```

```
    with torch.no_grad():
```

```
        output = model(  
            input_tensor  
        )
```

```
        probabilities = F.softmax(  
            output,  
            dim=1  
        ).cpu().numpy().flatten()
```

```
        predicted_label = int(  
            np.argmax(probabilities)  
        )
```

```
        display_image = (  
            image  
            .permute(1, 2, 0)  
            .numpy()  
        )
```

```
        mean = np.array(MEAN)  
        std = np.array(STD)
```

```
        display_image = (  
            display_image  
            * std
```

```

    + mean
)

display_image = np.clip(
    display_image,
    0,
    1
)

fig, axes = plt.subplots(
    1,
    2,
    figsize=(11, 4),
    gridspec_kw={
        "width_ratios": [
            1,
            1.7
        ]
    }
)

axes[0].imshow(
    display_image
)

axes[0].axis("off")

if predicted_label == true_label:
    title_color = "green"
else:
    title_color = "red"

axes[0].set_title(
    f"Настоящий класс: "
    f"{CLASSES[true_label]}\n"
    f"Ответ сети: "
    f"{CLASSES[predicted_label]}",
    color=title_color
)

colors = []

for i in range(10):

    if i == true_label:
        colors.append("green")

    elif i == predicted_label:
        colors.append("red")

```

```

else:
    colors.append(
        "steelblue"
    )

axes[1].barh(
    CLASSES,
    probabilities * 100,
    color=colors
)

axes[1].set_xlabel(
    "Вероятность, %"
)

axes[1].set_xlim(
    0,
    100
)

axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

for _ in range(3):
    show_test_example()

def predict_custom_image(
    path_or_pil
):
    model.eval()

    if isinstance(
        path_or_pil,
        str
    ):
        image = Image.open(
            path_or_pil
        ).convert("RGB")

    else:
        image = (
            path_or_pil
            .convert("RGB")
        )

    preprocessing = (

```

```

transforms.Compose([
    transforms.Resize(
        (32, 32)
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        MEAN,
        STD
    )
])
)

```

```

input_tensor = (
    preprocessing(image)
    .unsqueeze(0)
    .to(device)
)

```

```

with torch.no_grad():

```

```

    output = model(
        input_tensor
    )

```

```

    probabilities = F.softmax(
        output,
        dim=1
    ).cpu().numpy().flatten()

```

```

predicted_label = int(
    np.argmax(probabilities)
)

```

```

fig, axes = plt.subplots(
    1,
    2,
    figsize=(11, 4),
    gridspec_kw={
        "width_ratios": [
            1,
            1.7
        ]
    }
)

```

```

axes[0].imshow(
    image
)

```

```

axes[0].axis("off")

axes[0].set_title(
    f"Ответ сети: "
    f"{CLASSES[predicted_label]}"
)

colors = []

for i in range(10):

    if i == predicted_label:
        colors.append("red")

    else:
        colors.append(
            "steelblue"
        )

axes[1].barh(
    CLASSES,
    probabilities * 100,
    color=colors
)

axes[1].set_xlabel(
    "Вероятность, %"
)

axes[1].set_xlim(
    0,
    100
)

axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

print()

print(
    "Результат проверки "
    "своего изображения"
)

print(
    f"Определённый класс: "
    f"{CLASSES[predicted_label]}"
)

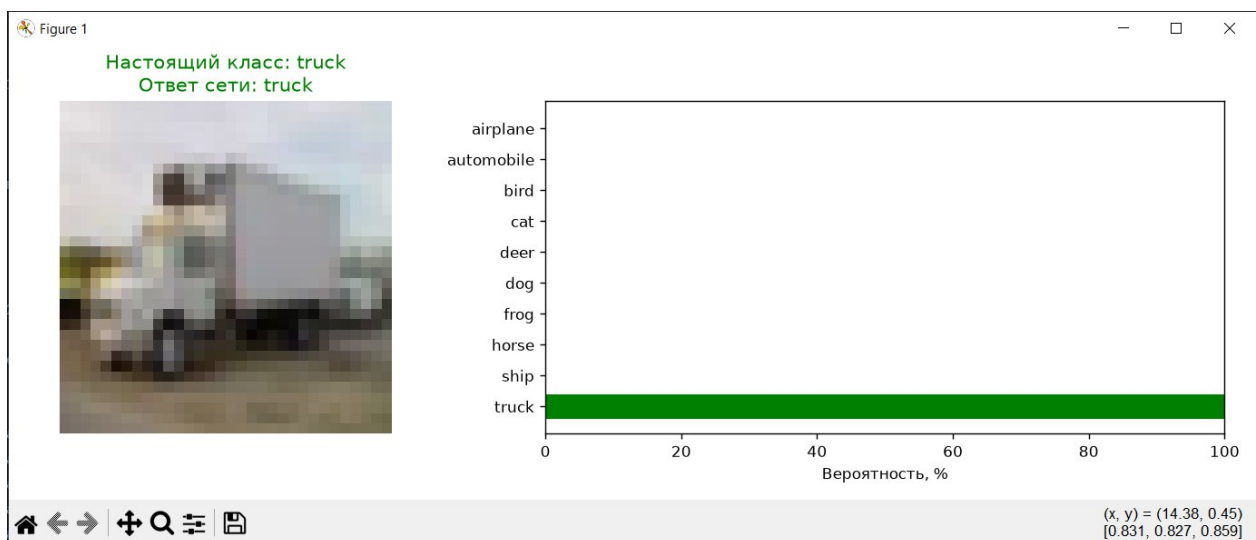
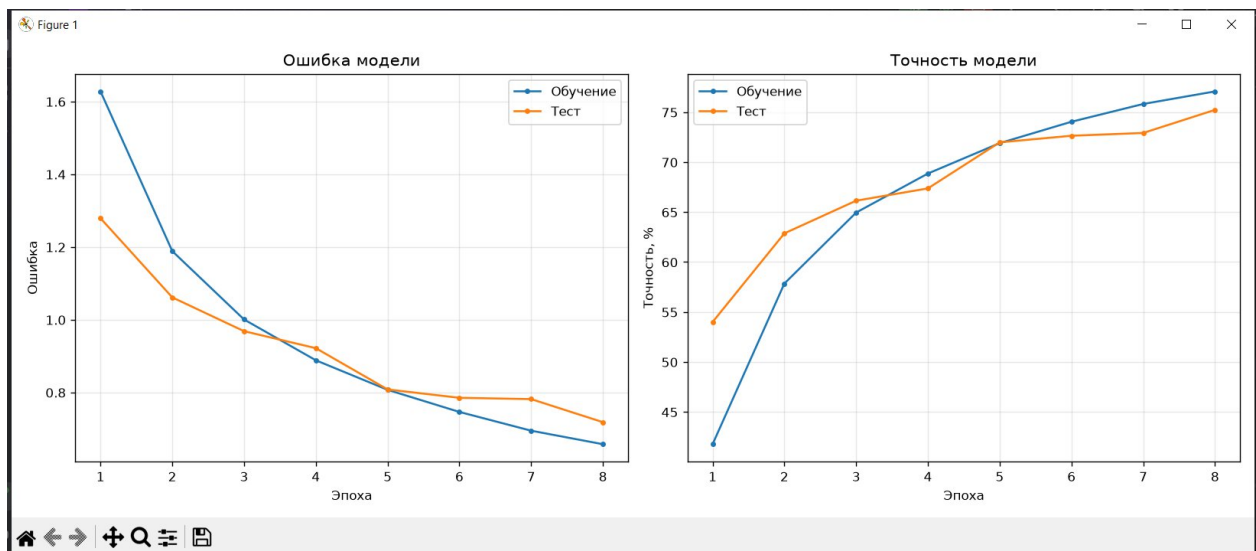
```

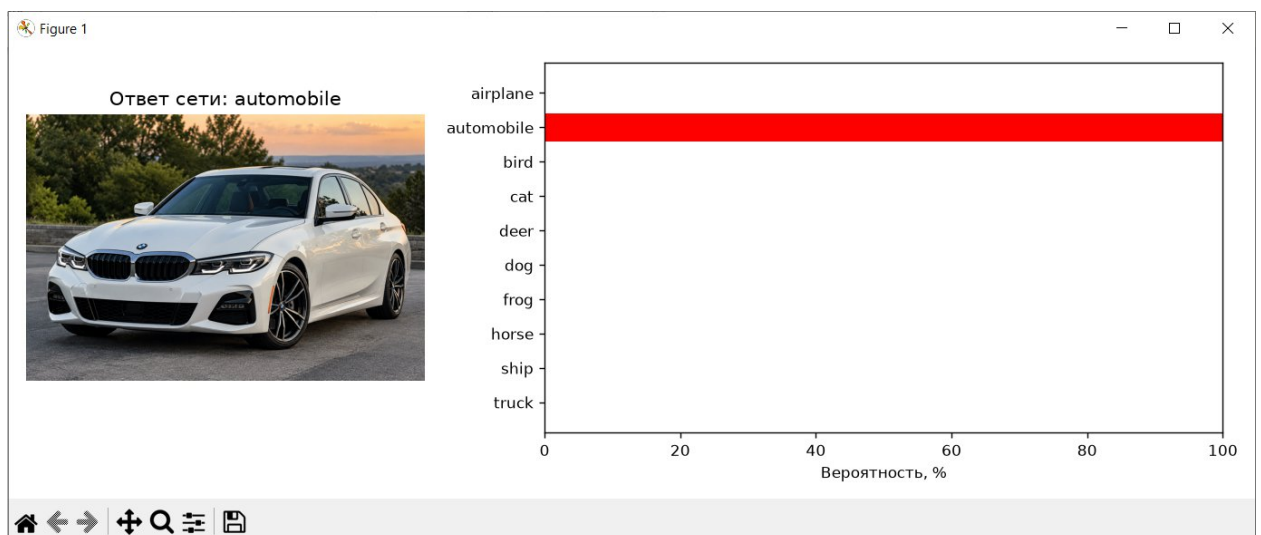
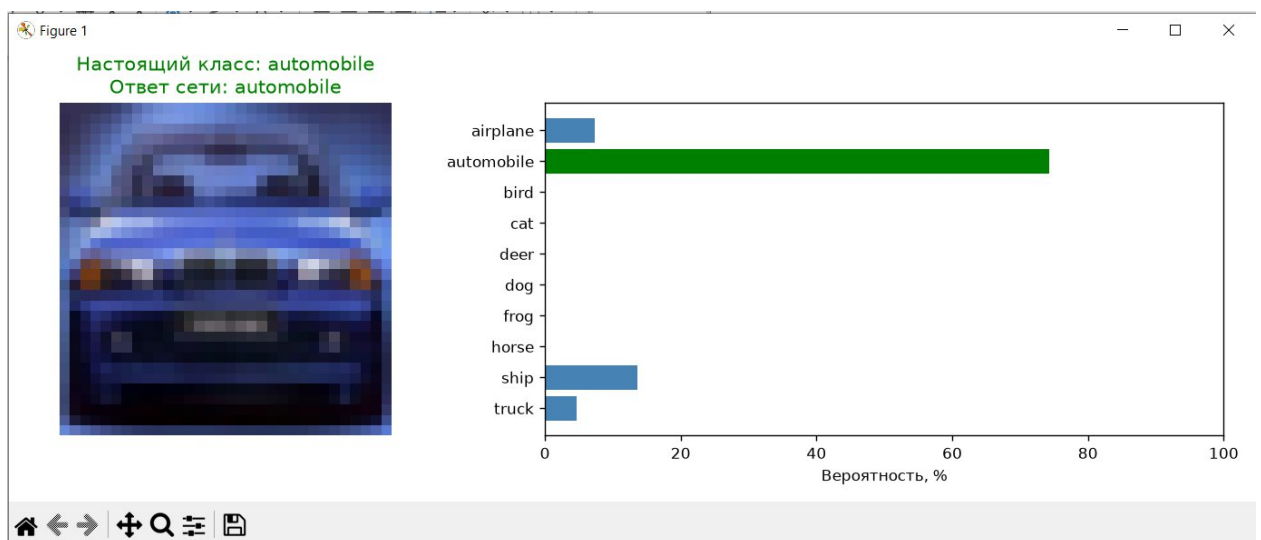
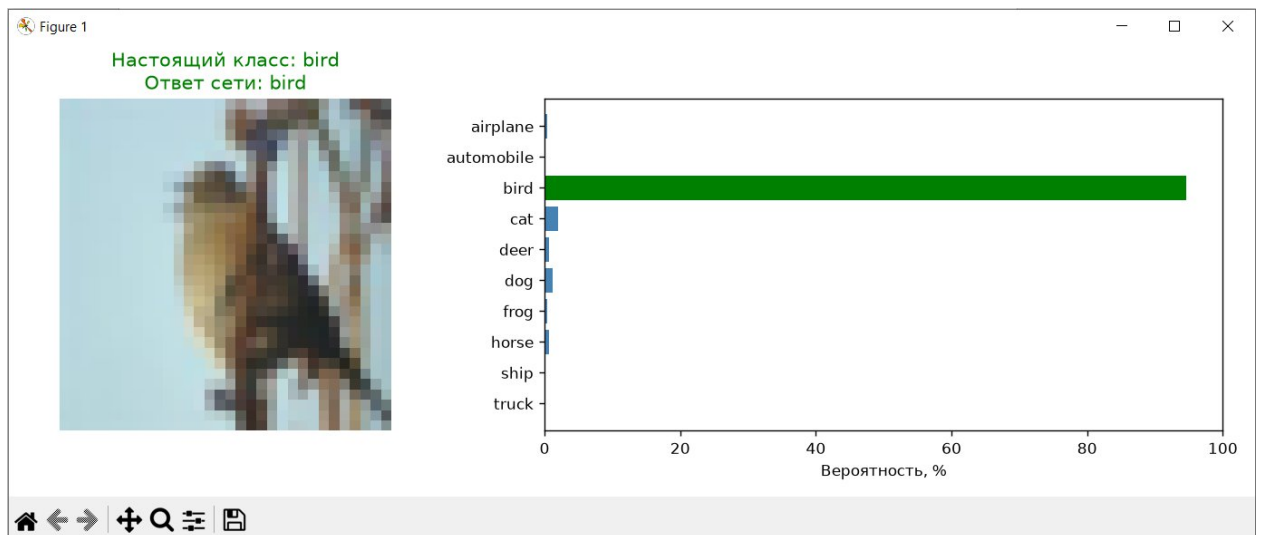
```
print(
    f"Уверенность модели: "
    f"{probabilities[predicted_label] * 100:.2f}%"
)
```

```
return (
    predicted_label,
    probabilities
)
```

```
predict_custom_image(
    r"C:\Users\Владислав\Desktop\ООИС\lab1syhovilo\car.png"
)
```

Результат программы :





Параметры программы

Устройство: CPU

Обучающая выборка: 50000 изображений

Тестовая выборка: 10000 изображений

Размер изображения: 32 x 32

Количество каналов: 3

Архитектура нейронной сети

Вход 3x32x32

Conv2d 32

Conv2d 64

Conv2d 128

Полносвязный слой 256

Выход 10 классов

Обучаемых параметров: 620362

Обучение нейронной сети

Эпоха 1 из 8 Ошибка обучения 1.6278 Точность обучения 41.77% Ошибка теста 1.2790
Точность теста 54.00%

Эпоха 2 из 8 Ошибка обучения 1.1886 Точность обучения 57.85% Ошибка теста 1.0614
Точность теста 62.87%

Эпоха 3 из 8 Ошибка обучения 1.0006 Точность обучения 64.95% Ошибка теста 0.9684
Точность теста 66.14%

Эпоха 4 из 8 Ошибка обучения 0.8884 Точность обучения 68.86% Ошибка теста 0.9217
Точность теста 67.37%

Эпоха 5 из 8 Ошибка обучения 0.8072 Точность обучения 71.90% Ошибка теста 0.8082
Точность теста 71.98%

Эпоха 6 из 8 Ошибка обучения 0.7464 Точность обучения 74.04% Ошибка теста 0.7852
Точность теста 72.64%

Эпоха 7 из 8 Ошибка обучения 0.6948 Точность обучения 75.82% Ошибка теста 0.7816
Точность теста 72.92%

Эпоха 8 из 8 Ошибка обучения 0.6576 Точность обучения 77.08% Ошибка теста 0.7182
Точность теста 75.23%

Результат обучения

Итоговая точность на тестовой выборке: 75.23%

Точность распознавания классов

airplane: 81.20%

automobile: 84.70%

bird: 73.40%

cat: 50.50%

deer: 65.80%

dog: 64.00%

frog: 75.10%

horse: 84.70%

ship: 87.80%

truck: 85.10%

Результат проверки своего изображения

Определённый класс: automobile

Уверенность модели: 99.95%

В ходе лабораторной работы была обучена свёрточная нейронная сеть для классификации изображений набора CIFAR-10 с использованием оптимизатора RMSprop. За 8 эпох точность модели на тестовой выборке выросла с 54,00% до **75,23%**, при этом точность на обучающей выборке составила **77,08%**. Небольшая разница между обучающей и тестовой точностью показывает, что модель не имеет выраженного переобучения и достаточно хорошо обобщает полученные знания на новых данных.

Наиболее высокая точность была получена для классов **ship — 87,80%**, **truck — 85,10%**, **automobile — 84,70%** и **horse — 84,70%**. Наибольшие трудности возникли при распознавании класса **cat — 50,50%**, а также классов **dog — 64,00%** и **deer — 65,80%**. Это связано с тем, что некоторые объекты CIFAR-10 имеют похожие визуальные признаки, а исходные изображения имеют небольшое разрешение 32×32 пикселя.

Полученная точность **75,23%** ниже результатов более сложных современных нейросетевых архитектур, применяемых для CIFAR-10, однако для относительно простой CNN, состоящей из трёх свёрточных слоёв и одного полносвязного слоя, результат является хорошим. Более сложные модели используют значительно больше слоёв, дополнительные методы нормализации, регуляризации и расширения обучающих данных.

При проверке собственного изображения автомобиля сеть правильно определила класс **automobile** с уверенностью **99,95%**, что подтверждает способность обученной модели распознавать изображения, не входящие непосредственно в тестовую выборку CIFAR-10.

Таким образом, разработанная CNN успешно справилась с поставленной задачей классификации изображений CIFAR-10. Использование оптимизатора **RMSprop** обеспечило стабильное уменьшение ошибки в процессе обучения, а полученные результаты показывают, что даже относительно простая архитектура нейронной сети способна достаточно эффективно решать задачу многоклассовой классификации изображений.

Вывод: Прodelав данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения